SYSTEM GUIDE

# Bitcoin-backed USDC one-loan system

The complete System Guide of Bitcoin-backed USDC one-loan system: the architecture guide, followed by all five domain guides.


Architecture guide: How the Bitcoin-backed USDC one-loan system works

Domain guide: Loan Terms And Commitments

Domain guide: Bitcoin Authorization And Recovery

Domain guide: Base USDC Settlement And Funding

Domain guide: Base Observation And Fallback Review

Domain guide: Participant Access And Loan Operations



Generated 15 October 2026 from the maintained guide sources. Source fingerprint 1ebc0f513d6c.

# Contents

**Architecture guide: How the Bitcoin-backed USDC one-loan system works** **3**
What the system is for 3
The five domains and their roles 4
A loan from offer to repayment 5
What binds everything to one loan 8
Who decides what, and what counts as proof 9
When things go wrong 10
What has been built and what is claimed 14
Choices and their reasons 14
Open questions 15

**Domain guide: Loan Terms And Commitments** **16**
One Responsibility, Shared Meaning 16
Boundaries And Exceptions 17
Choices And Scope 17

**Domain guide: Bitcoin Authorization And Recovery** **18**
Construction And The Two Participants 18
Package Lifecycle And Recovery 19
Fees, Failures, And Recorded Choices 19

**Domain guide: Base USDC Settlement And Funding** **21**
Settlement And Lender Interaction 21
From Funding To Repayment 21
Failure, Authority, And Scope 22

**Domain guide: Base Observation And Fallback Review** **24**
Establishing a fact the rest of the system can use 24
When the borrower repays but the lender does not claim 25
Limits and remaining recovery paths 26

**Domain guide: Participant Access And Loan Operations** **27**
Access, Coordination, And Local Composition 27
Cooperation Through A Loan 28
Payment Authority And Availability 28
Recorded Choices And Scope 29

**Technical records** **30**

ARCHITECTURE GUIDE

# How the Bitcoin-backed USDC one-loan system works

The system coordinates one loan of USD Coin (USDC) on the Base network between one borrower and one selected lender, secured by bitcoin that the borrower locks in a Bitcoin escrow. It is built so that each asset moves only through fixed routes that the contract rules and the participants' own signatures allow (who controls upgrades to the Base contract is one of the open questions), and so that the product and the organization running it never hold lender capital, a collateral key or a participant's unrevealed secrets.

## What the system is for

The product goal is simple to state: a borrower pledges bitcoin, receives USDC from a lender, repays, and gets the bitcoin back. The difficulty is that the loan lives on two chains that cannot see each other. Base records cannot spend or confirm Bitcoin, and Bitcoin cannot inspect Base repayment. The design keeps the two sides consistent through fixed, pre-authorized routes, and keeps authorities separate so that no term record, product identity, participant secret, observer, external provider or chain fact acquires another boundary's authority.

### Who takes part

- The **borrower** stages bitcoin as collateral, accepts the funded principal, and repays the exact debt.
- The **selected lender** publishes a firm offer for one loan and later funds it with an explicit action from a wallet or treasury they control; the product cannot pool, hold, substitute or automatically deploy that capital.
- Each participant has a dedicated **Vault**: a separate browser application that owns the participant's vault password, seed, per-loan Bitcoin keys and role secrets, and does its decryption, review and signing inside its own browser origin. Signing in to the product only gives access to encrypted data; it is not decryption, signing, secret recovery or chain authority.
- The **organization** operates the product, the application service and the Bitcoin automation. It has no key in the collateral script. Its Bitcoin fee wallet may fund a child transaction that adds fee to an already signed transaction (child-pays-for-parent, CPFP, through a pay-to-anchor output, P2A) without changing that signed transaction, and its Base fee wallet may submit permissionless fixed-effect calls, without gaining recipient, capital, debt, secret or contract authority.

- An **evidence-conditioned validation service (ECVS)** is a selected design for an independent service that, for one narrow failure case described under When the lender stays silent after valid repayment, could publish a per-loan secret. Its production availability remains gated: counsel approval and implementation evidence must activate it before any loan may commit to it or present it as available.
- A **developer** runs the whole system locally; that local demonstration is the target outcome of the current design and is still in progress (see What has been built and what is claimed).

### Concepts you need

- **Prefund and escrow.** The borrower first sends bitcoin to a prefund output. It becomes the loan's escrow only when an activation transaction spending it reaches six confirmations. Staged collateral, package preparation, Base acceptance and active collateral are kept as separate states and never collapse into one.
- **Packages.** Apart from the borrower's prefund, which she signs alone, and a cooperative settlement, which both participants sign freshly when they agree, each movement of the collateral uses a package: a transaction the participants signed before it is needed, stored as a partially signed Bitcoin transaction (PSBT) with a manifest. Each pays a fixed destination, so completing and broadcasting it later needs only the matching public condition, such as a revealed secret or a matured timelock. Because fees cannot be changed after signing, each package is signed in six fee variants.
- **Three secrets.** `X` is the borrower's acceptance secret, `Y` is the lender's claim secret, and `S` is the per-loan secret the ECVS could publish, only for a loan created after the fallback's production activation with the hash of `S` committed in its terms. Revealing `X` on Base accepts the loan and selects one fixed activation; revealing `Y` or `S` satisfies the same fixed borrower-recovery transaction. A public secret is fixed-route execution capability, not arbitrary signing, destination selection or chain authority.
- **Deadlines.** `D` is the repayment deadline. The lender's claim cutoff is `C = D + 172,800` seconds (48 hours). The lender's timelocked Bitcoin route becomes valid after `T = D + 259,200` seconds (72 hours). Bitcoin maturity is measured by consensus time (median time past, MTP), never by an application or Base clock.
- **Events are not outcomes.** A command can be accepted, a job completed, a transaction submitted, a Base event observed or finalized, and a Bitcoin transaction broadcast or confirmed. These are different facts, and the system reports each separately rather than one generic completion.

## The five domains and their roles

The system is divided into five domains. The separation matters because each can report progress without acquiring another's authority: login is not a Vault unlock, an accepted command is not a chain fact, and a Base refund is not a confirmed Bitcoin recovery.

| Domain | Its role and boundary |
|---|---|
| **Loan Terms And Commitments** | Owns the canonical meaning and immutable identity of the one-loan offer, the borrower reservation, the final terms and the environment-wide deployment facts. The binding grants no signing, secret-disclosure, funding, payout-selection, chain-observation or execution authority. |
| **Bitcoin Authorization And Recovery** | Owns participant-controlled Bitcoin keys and the prefund, escrow, activation, refund, recovery, default, cooperative-settlement, fee, relay and observation mechanisms, without giving the product, Base, the ECVS or the organization collateral authority. |
| **Base USDC Settlement And Funding** | Owns the Base loan record and the immutable per-loan funding and repayment vaults: selected-lender funding, exact repayment, the lender's claim and fixed-recipient refunds, without becoming a Bitcoin executor or a custodian of lender capital. |
| **Base Observation And Fallback Review** | Decides which Base events count and when they are final, with provenance, correction and replay, and holds the conditional ECVS boundary. It excludes Base execution, Bitcoin signing, destination choice and discretionary evidence arbitration. |
| **Participant Access And Loan Operations** | Owns product accounts, roles and access, the durable loan workspace with its commands, jobs, projections and package archive, the hand-offs to each Vault (expiring pointers, not signing authority), and the isolated local demonstration. It coordinates already-authorized actions and owns no participant private authority, no lender capital and no independent Bitcoin or Base truth. |

Every domain identifies the loan through the binding that Loan Terms And Commitments defines. Participant Access And Loan Operations carries each narrowly bound step to the participant's Vault and to the Base contracts. A hand-off to the Vault is an expiring pointer, not authentication, transaction consent or cryptographic authority, and the Vault never signs bytes supplied only by the product page. Base Observation And Fallback Review turns contract events into finalized facts that the workspace and the Bitcoin automation may rely on, and Bitcoin Authorization And Recovery takes a Base fact into account only once it is finalized, relying on its own independent observation for Bitcoin facts.

## A loan from offer to repayment

The following case is illustrative: Ana borrows, Leo lends, and everything goes to plan. The exceptions follow in When things go wrong.

### Agreeing the loan

Leo publishes a firm one-loan offer, which borrowers can browse. Ana selects it, and in one atomic step the system reserves the offer for her for 24 hours, in which she must produce a valid prefund, creates the stable loan identity and records the commercial draft; a concurrent attempt loses without creating a second reservation. Reservation does not fund USDC, move bitcoin or create signing authority. What must happen next before the terms are binding is an open question in the records. The architecture record says that publication is the lender's standing acceptance and that reservation creates the canonical loan without a second lender approval; the offer-and-reservation interface, the exclusion record for multi-lender matching and the offer selection also say there is no second lender approval. The loan-terms responsibility says that

neither publication nor reservation approves or commits the final bound terms, and it, the terms interface and the origination flow have Ana and Leo each explicitly approve the same final candidate, with the second matching approval committing it exactly once. Neither side is settled until the user answers.

While the reservation holds, each Vault allocates a fresh per-loan Bitcoin key, confirms that role's payout destinations, and generates the role's secret if it does not exist yet: `X` for Ana, `Y` for Leo. Each Vault returns only public keys, addresses and secret hashes; signing in, choosing destinations or acknowledging these results is not secret disclosure, approval, decryption or signing authority. In the origination flow's ordering, which is one side of that undecided question (the other holds that reservation already created the loan without a second lender approval), the complete final candidate is assembled from these results, and committing it produces the immutable terms and their hash; approval grants no signing or custody authority, and a change to the terms invalidates earlier approvals.

### Staging the collateral

Ana broadcasts the exact prefund. Once the terms are committed and the prefund is observed, even with zero confirmations, the Bitcoin domain builds six activation candidates and six refund candidates for it. Ana signs all twelve; Leo verifies the six activation candidates and adds his signature to them. The twelve signed transactions, the witness script, the manifests and their checksums are written as immutable records to the primary archive in Cloudflare, and the archive receipt is validated; an encrypted backup outside Cloudflare follows on a schedule, and a pending or failed backup raises an alert without blocking the loan. The attempt qualifies for funding only after six unchanged confirmations, fresh agreement between independent observers, and with at least 67 hours left before the borrower's prefund-refund route matures. A zero-confirmation prefund is not collateral, and qualification moves no USDC, reveals no secret and creates no escrow.

### Funding and acceptance

A narrowly authorized opener opens the Base funding window and deploys an immutable funding vault for this loan. After his explicit funding approval, Leo transfers USDC to it from any wallet or treasury he controls, and anyone may then finalize the vault once it holds at least the principal; exactly the principal is taken. A transfer, a balance or a submitted transaction is only provisional: funding counts when the funding event reaches Base layer-1 (L1) finality on Ethereum, and the sender gains no loan role.

Ana then accepts in her Vault, in two confirmations. She first confirms a review of the nonsecret preparation; the Vault then minimally unlocks `X` and her Base key and freezes the exact acceptance transaction, and a second confirmation bound to those exact bytes lets the Vault sign and submit it. The first confirmation is not consent to sign, a failure or cancellation clears the plaintext, and the application never receives `X` or signs the call. Acceptance needs funding no older than 24 hours and at least 19 hours left before the prefund refund matures. On acceptance the contract atomically registers a separate repayment vault, pays the principal to Ana's committed recipient and makes `X` public. Base acceptance is not Bitcoin activation: from here

until the expected escrow output has six confirmations, the loan is activation pending, and if one of Ana's prefund-refund transactions confirms first, Leo's principal is left unsecured (see When Base pays but Bitcoin activation does not confirm).

### Activating the escrow

Once `X` is final on Base, the platform selects one activation candidate, inserts `X`, and broadcasts only that candidate. The other signed candidates remain valid competitors until one of them spends the prefund, and whichever actually reaches six confirmations becomes the escrow; if one of Ana's prefund-refund transactions confirms first instead, no escrow exists and Leo's principal is unsecured. Once an escrow has six confirmations, the Bitcoin domain builds the terminal packages for it: Ana's normal-recovery variants and Leo's lender-recovery variants, each paying its signer's fixed destination. Whether Ana's long-stop variants are also built and signed in this phase is open, because the records disagree whether it holds twelve or eighteen packages (see Open questions). Each participant signs only their own routes. Whether the long-stop protects a borrower after valid repayment is undecided: a build constraint requires every new escrow to include this route, while the missing-secret security record says a loan without the activated fallback has no borrower fallback Bitcoin route (see When the lender stays silent after valid repayment). When these are stored and the archive receipt validates, the loan is package-ready active. Until then it stays explicitly terminal-signing-pending; that state keeps prompting the affected participant but cannot reverse the paid principal or the confirmed escrow, and it must not be presented as package-ready.

### Repaying and closing

Before `D`, anyone may transfer the exact debt to the repayment vault, including from a wallet that is not Ana's; paying grants no loan role, refund right or destination control. Anyone may then finalize the repayment, which is valid only when the exact debt from the registered vault is final on Base no later than `D`. Leo claims strictly before `C`: after fresh reauthentication and multi-factor authentication, he first confirms a review of the nonsecret claim; his Vault then unlocks `Y`, and a second confirmation bound to the exact bytes lets it sign and submit the claim; a failure or cancellation clears the plaintext. The contract pays the exact debt once to Leo's fixed recipient and makes `Y` public. Once `Y` is final on Base, the platform inserts it into Ana's already signed recovery transaction and broadcasts it, with no fresh signature and no changed output. The Bitcoin side is complete only when that spend confirms; the application does not report completion before confirmation.

The figure condenses the case. Each hand-off between the chains waits for a final fact on the other chain, and the Base and Bitcoin steps are never one atomic transaction; between acceptance and activation, Leo bears the risk that Ana's prefund refund confirms first:

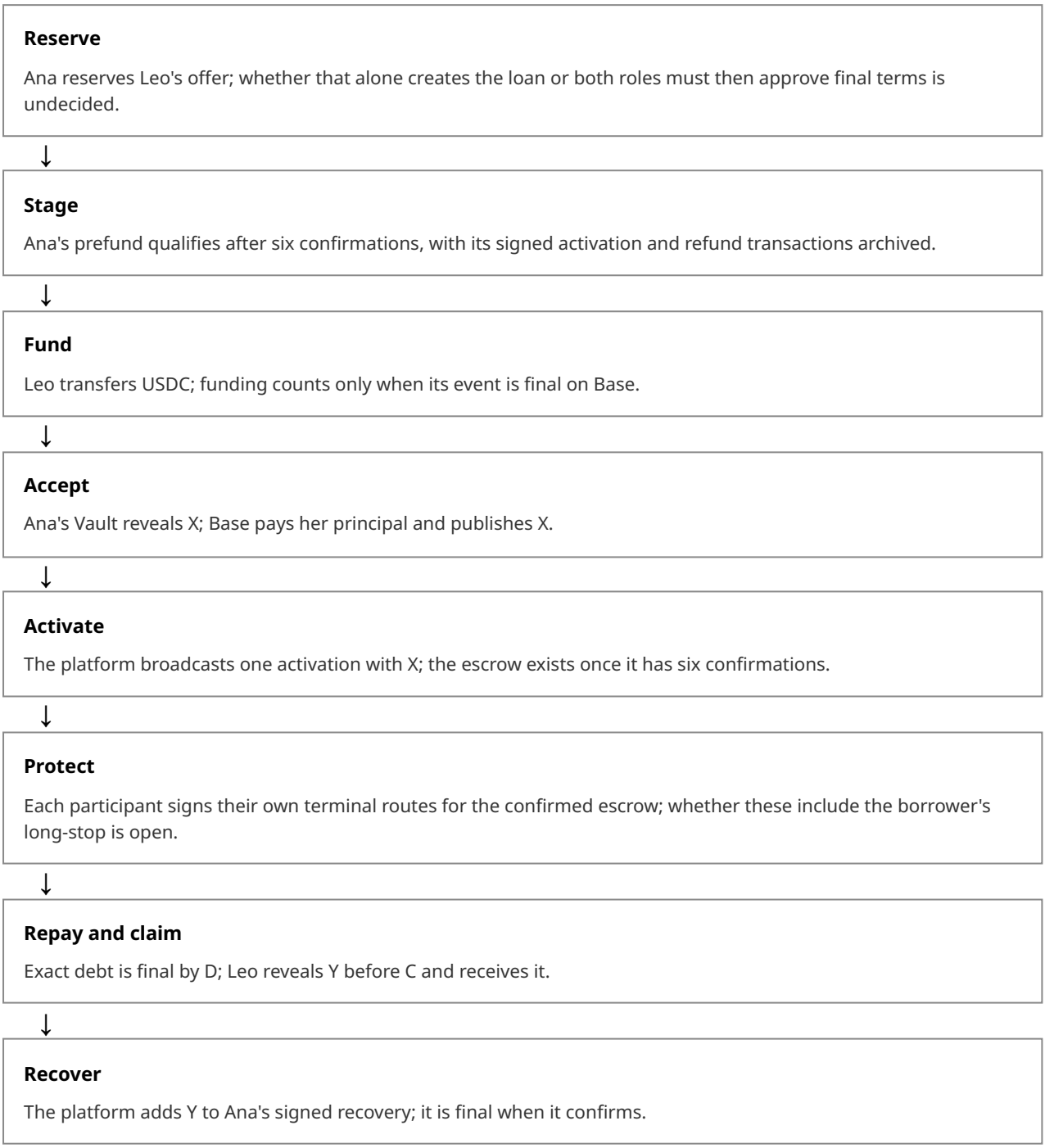



Figure 1. Base acceptance and Bitcoin activation are separate steps; the borrower's bitcoin is back only when the recovery transaction confirms.

## What binds everything to one loan

Every domain identifies the same loan through one canonical terms value, `LoanTermsV1`, and its terms hash. The binding covers the two participants, asset and amount terms, deadlines, the loan-specific Bitcoin public keys, independently selected external Bitcoin destinations, Base

protocol addresses and fixed settlement recipients, and the commitments to `X`, `Y` and the conditional `S`. Whether it also carries the selected offer revision and reservation number is unresolved: the architecture record and the loan-terms responsibility say it does, while the terms interface says offer and reservation identity belongs to the offer-and-reservation contract, not the terms value. A separate deployment manifest, `ProtocolDeploymentV1`, binds environment-wide facts: chain, contracts, asset, Bitcoin network, timing policy and upgrade authority. Neither commitment is a wallet, a signing key, a source of funds, a chain-state fact or a lifecycle action. Any Base record, Bitcoin artifact, workspace record, observation or secret commitment that names the wrong loan, chain, asset, key, destination or version is rejected, and no workspace copy or acceptance snapshot can redefine committed terms.

A participant's Vault payment account is a separate public source-of-funds identity, distinct from the protocol account that makes the participant's Base calls, from every loan key and from every organization wallet. Selecting its address as a fresh loan's existing recipient value requires fresh approval and does not rewrite historical terms. The capability's delivery status is itself an open question: the payment selection records it as partial, with implementation and execution evidence still gated, and the local-demo browser verification records it as planned and unverified, while the core coordination hand-off says historical payment implementation is already recorded and the hand-off sequence records a correction that passes focused probes without finalized receipts. At most it is a selected design whose implementation and verification are unconfirmed.

## Who decides what, and what counts as proof

Authority is split so that no record, identity, secret, observer, provider or chain fact acquires another boundary's authority:

- **Loan meaning.** Loan Terms And Commitments owns the canonical meaning, the offer and reservation identity, the terms versions and the binding.
- **Participant secrets and signatures.** Each participant's Vault owns the password, seed, loan keys and unrevealed `X` or `Y`, and signs the participant's own Base calls and Bitcoin packages. The product, the service, operators, logs and the package archive never receive them.
- **Capital.** The selected lender owns capital approval and funding. The product presents the immutable funding vault and may accept a participant's Vault payment account as one eligible payer, but it cannot pool, hold, substitute or automatically deploy lender capital. Whether that payment capability is only selected with its evidence gated or already implemented is undecided between its records.
- **Base state.** The Base contracts own funding, acceptance, repayment, claim, fixed-recipient refund, deadlines and the public `X` and `Y`. Raw USDC transfers, vault balances, submitted transactions and application status remain provisional until the named contract event reaches the approved finality boundary.
- **Bitcoin state.** Bitcoin consensus owns the prefund and escrow outputs, signatures, timelocks, broadcasts, confirmations, reorganizations and the one final spend of the escrow.

- **Observation.** The Base observer reports an authoritative fact with its provenance and finality; it does not create the fact or approve its consequence. Independent Bitcoin observers do the same for Bitcoin and never construct, relay, price, sign or authorize Bitcoin actions.
- **Coordination.** The loan workspace owns durable commands, the job outbox, sourced projections, alerts, package custody and the coordination of already-authorized actions, and owns no participant private authority, no lender capital and no independent Bitcoin or Base truth.
- **The conditional ECVS.** For a loan created after its production activation with the hash of `S` committed, it may verify the exact finalized Base refund event and publish only that loan's committed `S`. It is not an evidence arbiter, a Bitcoin signer, a destination selector or a discretionary release authority, and its production availability remains gated.
- **The local demonstration.** Its composition owns only the explicit local-demo profile and cannot redefine protocol meaning, become a chain source of truth or activate a public profile.

What counts as proof differs by chain. The application acts on a Base fact only after the block containing it reaches Base L1 finality; it may display an event earlier, labeled as awaiting finality, but creates no lifecycle transition, Bitcoin action or completion status from it. Base L1 finality is chosen as one consistent rule at every consequential cross-chain boundary, accepting extra waiting rather than promising an exact waiting time. On the Bitcoin side, a relay's response proves nothing; independent observation owns mempool and confirmation facts, and only a confirmed spend settles an outcome. When `X` or `Y` becomes final on Base, or `S` is validly published, the secret permits only the already-authorized matching Bitcoin route: Base facts and public secrets never sign, redirect, broadcast or confirm Bitcoin by themselves.

## When things go wrong

### When origination stops before acceptance

If funding or acceptance expires, or the attempt is cancelled or fails before `X` is final on Base, the two sides unwind separately. Anyone may permissionlessly return Leo's principal, and any excess or late USDC, only to his fixed lender recipient. Ana's bitcoin returns through one of her signed prefund-refund transactions once its relative timelock matures, the prefund is unchanged and unspent, two observers agree, and Base shows no accepted `X`; the application cannot satisfy that gate from a wall clock, a stale source or a relay response. There is no company-held `X`, delegated acceptance or company activation authority: when the borrower does not accept the funded principal, funding expires and the loan unwinds this way, and automatic activation recovery after borrower nonresponse is deferred. Once `X` is final, this unwind is forbidden and activation recovery continues instead.

### When Base pays but Bitcoin activation does not confirm

Base acceptance and Bitcoin activation are not one atomic transaction. After `X` is final, recovery must keep pursuing the same selected activation; it cannot refund the prefund, pay principal again or construct an unapproved replacement. If activation is delayed long enough that the borrower's prefund refund matures and confirms first, Ana keeps both the principal and her

bitcoin, and Leo is left with unsecured credit exposure. This is the principal residual cross-chain risk, and it is the lender's to bear: no component may claim literal atomicity or mark the loan active before the expected escrow has six confirmations.

### When the lender stays silent after valid repayment

Suppose Ana's exact repayment is final on Base by `D`, but Leo never claims before `C`, because he lost access to his Vault, is unavailable, or withholds `Y`. Missing `Y` alone is not the trigger and lender silence is not borrower default; at `C` the contract permanently disables the claim and freezes Ana's refund recipient, and anyone may then trigger a fixed-effect refund of the exact debt to that recipient. Before `C`, Ana's committed protocol account may irreversibly switch that recipient to itself. Delayed automation does not expire the refund right, and once the refund is final Leo can never also collect the USDC. The refund does not depend on the ECVS, which must never gate it, and it does not by itself unlock the bitcoin.

The bitcoin, however, stays in escrow, because Bitcoin cannot see the Base refund. What can happen to it next depends on records that disagree in part:

- The conditional ECVS route exists only for a loan created after its production-activation gate, with the secret's hash committed before the escrow was built. For such a loan the ECVS verifies the exact finalized refund event, publishes `S`, and the platform inserts `S` into the same borrower-signed recovery transaction; publication or broadcast alone is not recovery. Today this route is unavailable: counsel approval and implementation evidence have not activated it.
- Leo's lender-recovery package remains Bitcoin-valid after `T`, 24 hours after `C`, and pays his fixed destination. No hacker is required for the borrower to lose collateral after paying: Leo, or someone holding a copy of his signed package, can broadcast it; the holder cannot redirect the fixed payout, and the application cannot revoke a valid package held outside it.
- Both participants may agree to a cooperative settlement, as long as no other terminal spend has confirmed.
- The borrower long-stop is an open question. The long-stop constraint requires every newly constructed escrow to include borrower long-stop candidates, and the default-timing invariant makes such a package valid after `B = D + 864,000` seconds (ten days) without `Y` or `S`, racing the still-valid lender route until one confirms; the long-stop recovery flow owns that route. The security record on this case says neither the long-stop nor the `S` route is protection until implemented for the escrow and confirmed before the lender transaction, and whether the long-stop variants are among the signed terminal packages also turns on the open terminal-package count. The missing-secret security record says a loan without the activated fallback has no borrower fallback Bitcoin route, the residual-risk invariant requires documenting an unsupported borrower-Bitcoin outcome, and the unclaimed-repayment flow lists the remaining routes without the long-stop. Whether these describe different loans or contradict each other is undecided.

In every case Bitcoin settles on the one spend that confirms first, and the product may not claim that Ana's collateral is currently recoverable.

### When the borrower does not repay

If no exact repayment is final by D, the loan is classified as ordinarily defaulted for product routing. Missing Y, watcher silence, a raw transfer or an unavailable fallback is never substituted for missing repayment, and an observer conflict pauses the classification instead of inferring default. Leo's route still waits for Bitcoin consensus time to pass T. In this version the organization never schedules or broadcasts a default transaction: once the timelock condition is valid and after fresh reauthentication, Leo may export his current signed package and broadcast it himself. Anyone holding a copy can broadcast it too, but the payout goes only to Leo's fixed destination, and a copied package cannot be recalled by application status. It is final only when it confirms. Automatic organization broadcast is deferred. A cooperative settlement remains available until another terminal spend confirms.

### When both participants want to settle differently

Borrower and lender may agree on a different split of the unspent escrow. Each freshly signs the same six agreed variants with their existing loan key, and the platform may broadcast one variant once both signatures are stored and the archive receipt validates; one-sided consent, a lost vault password or any mismatch stops the attempt and leaves earlier routes intact. The Bitcoin mechanism is defined, but the signing-session experience, the matching Base accounting and support procedures are deferred, so the product does not offer this as a complete operational capability.

### When observers disagree or are unavailable

If Base observation sources conflict or are both unavailable, the system keeps its finalized projections and blocks only new Base-derived consequential actions until replay recovers; a reorganization correction rolls the current view back to the last common finalized block and replays history. If Bitcoin observers disagree or are unavailable, packages and authority are preserved, an alert is raised, and only the automation that depends on the contested fact stops, with no manual override; it resumes only on fresh agreement.

### When Cloudflare is down

All production application artifacts in this version run on Cloudflare: the product, the application interface with its D1 database, the Base observer, the Vault and the Bitcoin operations container. They keep separate builds, credentials, origins and releases, but they share one provider failure domain. A Cloudflare-wide outage can therefore suspend the product, the Vault, the application interface and database, Base observation, Bitcoin automation and the operator surface together. The chains continue, but the application accepts no new consequential command and claims no chain result. After Cloudflare returns, the system replays and reconciles Base and Bitcoin before resuming its pending jobs.

There is no live fallback on another provider, no active-active deployment, no guaranteed cold redeployment and no stated recovery time or recovery point objective. The encrypted backups outside Cloudflare protect the signed package data, so a provider or company disappearance need not force fresh signatures after a successful backup, but they are not a running service, and

the newest package set not yet backed up remains exposed to loss. An outage also has a cost beyond inconvenience: after a secret has become public, it can delay a required Bitcoin action and worsen a refund or terminal-route race.

### When a component or credential is compromised

No threat is summarized by a severity word; the analysis states the compromised capability, the exact lifecycle state, the direct consequence and the recovery opportunity. Combinations deserve more attention than isolated leaks, because independently insufficient capabilities can collapse into one shared failure domain or become useful at a later state:

- A thief of the signed package archive cannot change any committed destination or amount, but once the matching public secret or matured timelock exists can force that fixed route, including a default after valid repayment.
- A malicious shared frontend that captures both participants' keys at different moments is the dominant common-mode threat: once escrow exists, it can steal the collateral through the cooperative route, and it is more important than two unrelated account takeovers.
- A compromise of Base upgrade or administration authority could redirect or lock many users' USDC if upgrades could alter existing funded loans. Who controls upgrades in this version is undecided: some records keep a 2-of-3 Safe through a seven-day timelock, while the upgrade-governance selection gives a single key control during a short public mainnet beta (see Open questions).
- Product account takeover alone is not Vault decryption; it exposes metadata and supplies the ciphertext half of a possible offline password attack.

### What the system cannot guarantee

- It cannot make Base acceptance and Bitcoin activation atomic, so the lender bears the activation-pending risk described above.
- It cannot stop a lender, or a holder of the lender's signed package, from taking the collateral after the borrower repaid but the lender stayed silent, unless a borrower route confirms first; a holder can only send it to the lender's fixed destination, never redirect it; whether such a loan has a borrower long-stop route is undecided, since a build constraint requires every new escrow to include one and the missing-secret security record says a loan without the activated fallback has no borrower fallback Bitcoin route.
- It cannot recover a forgotten vault password. Product account recovery, support, ciphertext restore or company backup does not recover the vault's plaintext seed or secrets.
- It offers no live continuity during a Cloudflare-wide outage.
- It has no liquidation: selling bitcoin, returning surplus and enforcing shortfalls are outside this version, and so are public or multi-lender matching, pooled capital, product custody and automatic capital deployment.

## What has been built and what is claimed

The required outcome of this design is a developer-operated local demonstration: two isolated actors use the real product, application interface, Vault, Base contracts, observer, Bitcoin operations and protocol library through their public seams to complete one representative loan through normal repayment and confirmed borrower recovery. It follows the same journey described above; there is no demo-only lifecycle. Demo identities, tokens, wallets, balances, contracts and provider substitutes exist only inside that local profile, and any public credential, non-local chain or non-loopback setting makes it fail closed before provisioning.

That demonstration is the target, not a current capability. The hand-off sequence records the local-demo runtime as still in progress: its supervisor still contains an explicit not-implemented stop and its verification steps have not passed. The design also targets deterministic local verification, a separate claim that would prove neither a developer-runnable product nor a deployment; its verification records are planned and unverified. It makes no testnet or mainnet deployment claim and no activated-operation claim: those need their own later credentials, funding, deployment, monitoring, legal and release gates, and an intentional enablement action. A configuration name or a successful local run activates nothing.

## Choices and their reasons

- **Five domains.** The domains separate agreement meaning, participant Bitcoin authority, Base value movement, verified observation with the conditional fallback, and durable application coordination, so that progress in one never stands in for another's authority. The user approved adding Participant Access And Loan Operations as the product application layer separate from the protocol domains; the other four domains were chosen by the agent.
- **Base L1 finality before any consequential hand-off.** One uniform rule across funding, secret disclosure, repayment and refund, at the cost of roughly 20 minutes of waiting per hand-off, prevents a transient Base observation from triggering a cross-chain action.
- **One cloud provider.** Keeping every production artifact on Cloudflare, with separate accounts, projects and credentials where they limit blast radius, is an explicit simplicity trade-off accepted instead of multiplying failure domains before those operations are proven. It is not a continuity claim.
- **Pre-signed fixed-destination packages with fixed fee variants.** Participants authorize exact transactions in advance, so later execution needs only a public condition and a holder of a stored package cannot redirect value; fees are fixed because the transaction is signed before use.
- **Manual lender-default broadcast.** The security analysis records that leaving default broadcast to the lender reduces the organization's execution authority but gives the lender a direct ability to bypass application policy after maturity.
- **An ECVS without a Bitcoin key.** The selected fallback design gives the service no Bitcoin key and no discretionary judgment; its secret can still be disclosed too early, withheld, lost or compromised, which is why independent controls and the production-activation gate remain material. Only a loan created after that gate, with the hash of `S` committed in its terms, can use it.

## Open questions

The records leave these points undecided. Each side is stated as its record holds it; none is settled until the user answers.

- **Approval of final terms.** The architecture record says publication is standing acceptance and reservation creates the canonical loan with no second lender approval, and the offer-and-reservation interface, the multi-lender exclusion record and the offer selection also say there is no second lender approval; the loan-terms responsibility, the terms interface and the origination flow say both roles explicitly approve the same candidate and the second matching approval commits it. Neither is settled.
- **Fee when no signed tier qualifies.** The fee interface's payload says to select the 200 satoshis-per-virtual-byte tier, record CPFP eligibility and alert for a separate manual fee approval, and two ticket resolutions likewise use the 200 tier with CPFP eligibility and an alert; the same interface's tier-selection action says to fail closed and select nothing. Neither is settled.
- **Number of terminal packages.** One sentence of the package-construction responsibility and the durable package-archive constraint speak of a twelve-package terminal phase, while the package-construction responsibility's own construction statement, the manifest interface and the Bitcoin state records specify six borrower-recovery, six lender-recovery and six borrower long-stop variants, eighteen in all. Neither count is settled.
- **Offer identity in the terms value.** The architecture record and the loan-terms responsibility say `LoanTermsV1` carries the selected offer revision and reservation number; the terms interface says that identity belongs to the offer-and-reservation contract, not the terms value. The records do not declare this disagreement, and neither side is settled.
- **Base upgrade control.** The upgrade invariant, the immutable-conversion scope record and the settlement responsibility say this version keeps upgrade administration under a 2-of-3 Safe through a seven-day timelock; the upgrade-governance selection says a single upgrade key controls local, test and a short public mainnet beta before ownership moves to that timelock. Neither is settled.
- **On-chain pause.** The settlement responsibility, the upgrade selection and the window-opening constraint say there is no on-chain pause; the deployment and upgrade state record lists pause roles and a paused state. Whether that state is an application-level stop or an on-chain role is undecided.
- **Borrower long-stop after valid repayment.** As described under When the lender stays silent after valid repayment, the long-stop constraint requires a long-stop route at `B` in every new escrow, while the missing-secret security record says a loan without the activated fallback has no borrower fallback Bitcoin route. This is undecided.
- **Vault payment account status.** As described under What binds everything to one loan, the records disagree whether the payment capability is only selected and gated or already implemented. This is undecided; the guide treats it at most as a selected design.

The technical architecture record holds the exact purpose, binding, authority and boundaries, and each domain guide explains its domain in full.

DOMAIN GUIDE

# Loan Terms And Commitments

Every part of the product must refer to the same loan. A borrower and a selected lender can see an offer, authorize keys and recipients in their Vaults, move USDC on Base, and prepare collateral transactions on Bitcoin; those artifacts must agree about the participants, amounts, deadlines, destinations, and secret commitments. This domain owns that shared meaning. A matching identifier proves which agreement an artifact describes; it does not authorize a spend or establish a chain outcome.

A **firm offer** records the lender's proposed commercial values. A **reservation** selects one offer revision for one borrower and creates a stable random `loan_id`. **LoanTermsV1** is the ordered, versioned agreement data; its `terms_hash` binds the exact encoded values. **ProtocolDeploymentV1** is different: it identifies the approved environment, chain, contracts, asset, Bitcoin network, and policy configuration. URLs, credentials, secrets, and mutable application state are not part of that deployment identity. Separating immutable loan facts from environment facts prevents different consumers from signing or processing different meanings under the same label.

## One Responsibility, Shared Meaning

**Loan terms, one-loan offer, reservation, and parameter binding** is the domain's single responsibility. It defines identity, encoding, candidate versions, approvals, commitment, cancellation, and supersession. Its output is used by every other domain: Bitcoin construction binds the package graph to it; Base settlement accepts only the pinned terms and deployment; observers reject wrong-deployment evidence; participant operations persist and present the same candidate; Vaults check the same role and loan binding before returning public identities or signing. The loan terms and canonical binding contract, the protocol deployment manifest, and the offer, reservation, and loan binding contract own the exact representations and public boundaries.

The detailed records distinguish commercial reservation from final agreement. Reservation creates the loan identity before loan-specific Vault derivation. Both roles then supply durable acknowledged public allocation results. The borrower prepares its public destination and `H(X)` result; Core assembles a complete candidate version and hash. Each role explicitly approves that same candidate, and the atomic second matching approval commits it once. A hashed-field edit invalidates earlier candidate approvals. Duplicate identical approvals and commitment retries return the same result; stale, wrong-role, wrong-loan, mismatched, expired, or cancelled approvals are rejected. The offer and reservation states, the canonical loan terms states, and the origination flow preserve that ordering. The architecture record instead states that publication is the lender's standing acceptance of the offer and that reservation creates the canonical loan

without a second lender approval. This is an unresolved source conflict: neither wording is chosen until it is answered, so whether the lender approves the final candidate separately remains open.

## Boundaries And Exceptions

A protocol Bitcoin public key, Base protocol externally owned account (EOA), external payout destination, and Vault payment account are distinct identities. The payment account can fund an eligible transfer; it does not silently become a loan recipient or protocol caller. Selecting an address for a fresh loan's existing recipient field uses the normal role authorization and fresh candidate approvals. Historical terms remain immutable. Likewise, an acceptance snapshot or workspace copy cannot become a second authority for agreement meaning. The canonical loan binding, selected lender capital control, authority separation, and key, destination, and refund separation invariants make those boundaries explicit.

Every consumer rejects an unsupported encoding, changed fingerprint, wrong chain/contract/asset/network, mixed environment, wrong key or destination, or dependent artifact that does not bind exactly. Pre-activation amendment produces a later candidate/version and fresh dependent artifacts. Cancellation ends that reservation; reopening requires a fresh loan identity. These rules prevent an apparently successful retry from reviving old approval or signing material.

## Choices And Scope

The recorded rationale for canonical encoding is interoperability and verifiability: divergent encodings or mixed immutable and deployment facts could bind signatures, manifests, and chain actions to different loans, or make a package impossible to reproduce. The exact encoding and accepted address spellings remain in the loan terms and canonical binding contract; the domain explanation does not duplicate that byte-level contract.

The lender separately retains the capital-funding decision, even after an agreement is committed. Product authentication, Vault decryption, Bitcoin construction, Base execution, observation, and durable workflow coordination remain with their respective domains. Public or multi-lender matching, pooled capital, product custody, and automatic deployment of lender capital remain excluded under the matching, pooling, custody, and automatic capital exclusion, and liquidation under the liquidation, surplus, and shortfall exclusion. An unresolved implementation question belongs in its existing ticket; a narrative or diagram cannot supply a missing approval.

The architecture guide shows where this domain fits in the system, and the domain record lists exactly what the domain owns and excludes.

DOMAIN GUIDE

# Bitcoin Authorization And Recovery

This domain preserves participant control of Bitcoin collateral while making fixed, previously authorized transactions operable across interruptions. A **prefund** stages borrower Bitcoin before activation; an **escrow** is the later confirmed collateral output. A **PSBT** (partially signed Bitcoin transaction) carries a transaction and signing material for review and signature. A signed package is evidence of exact prior authorization, not permission for an application to choose a new destination. Bitcoin consensus, including confirmations and timelocks, remains the authority for spendability and the final outcome.

The secrets serve different actions: borrower `X` enables acceptance and fixed activation, lender `Y` enables normal borrower recovery after repayment claim, and optional independent `S` enables the same recovery transaction under the separate finalized-refund fallback. A long-stop is instead a participant-authorized absolute-timelock route without either recovery secret. None grants the organization a collateral key.

## Construction And The Two Participants

**Bitcoin prefund, escrow, and package construction** constructs and validates scripts, outpoints, amounts, fixed outputs, fee candidates, manifests, signatures, and the package graph. It operates only the route justified by current chain and package evidence. It consumes committed terms and acknowledged participant allocations; it cannot allocate a key or invent a missing binding. It also owns constrained fee selection, relay, independent chain observation/reconciliation, and the boundary around a separately funded fee-bump child.

**Borrower Bitcoin authorization and recovery** keeps the borrower Vault, seed, loan keys, `X`, and signing local to the dedicated Vault origin. It creates or restores the loan's immutable allocation, acknowledges the encrypted snapshot, prepares public destination/commitment material, and signs an exact canonical batch. Acceptance first reviews nonsecret facts; a narrowly unlocked worker prepares the exact secret-bearing transaction; a second confirmation tied to those bytes authorizes signing and submission. Password or login recovery cannot recreate a forgotten Vault password. **Lender Bitcoin authorization and recovery** applies the corresponding local boundary to the lender's key and `Y`, fixed default-package signing, normal release, manual export, and fresh cooperative signatures. Account access never exposes either participant's plaintext authority. The Vault envelope and local handoff, the participant-local Base calls, and the cooperative settlement package contracts retain these exact interactions.

## Package Lifecycle And Recovery

Origination can construct and sign against an exactly observed zero-confirmation prefund after its committed binding and preparation prerequisites, but six unchanged confirmations are required for funding-window opening, the pre-X gate, and reliance. The initial family contains six activation and six borrower refund candidates at 10, 20, 50, 100, 150, and 200 sat/vB (satoshis per virtual byte). Terminal packages are constructed only for the actual activation winner after six confirmations. The package contract specifies six borrower recovery, six lender recovery, and six borrower long-stop variants, while the construction responsibility's archive-readiness rule and the durable signed exit package archive constraint describe a twelve-package signed terminal phase. This source conflict is unresolved and remains for a separate repair; neither count is chosen until it is answered. The prefund and origination attempt states, the package graph states, the origination binding and package manifest contract, and the zero-confirmation boundary remain the exact owners.

The order is unsigned construction, immutable canonical signing batch, participant Vault signing, Core cryptographic validation and complete writes to the R2 package archive, then the receipt committed in the D1 workspace database. The receipt cannot be an input to the signing action that produces it. Incomplete terminal signing after activation leaves an explicit pending condition and continued recovery work; it cannot undo principal or escrow. A different activation winner requires fresh terminal construction/signing. A changed or lost parent recursively invalidates descendants; a reorg of the same unchanged parent preserves bytes and signatures but suspends qualification. This avoids treating stale transactions as executable.

Finalized public Y, or valid independently published S for a production-activated commitment, fills the same borrower-signed recovery transaction (route code 3) without changing its output, signature, ID, leaf, or manifest entry. Lender default and borrower long-stop have their own consensus deadlines and remain concurrent once both are valid; only one confirmed spend wins. Whether every loan has the borrower long-stop route is an open question: a build constraint requires every newly constructed escrow to include long-stop candidates that need neither Y nor S, and the default timing invariant and long-stop recovery flow define it, while the missing-Y security boundary says a loan without the production-activated S commitment has no borrower fallback Bitcoin route; the records do not settle whether these describe different loans or contradict each other. A Base repayment or refund cannot revoke those signatures. Cooperative signing requires fresh participant agreement and both matching loan keys, while its complete product/accounting operation remains deferred. The finalized secret to fixed Bitcoin action contract, the escrow and terminal outcome states, the one terminal escrow outcome invariant, and the missing-Y residual lender risk preserve the exact boundary.

## Fees, Failures, And Recorded Choices

Fixed fees are necessary because the parent is signed before use. Provider-neutral integer units, actual virtual size, strict buffer comparison, estimate freshness, and replacement rules prevent underestimation or unsafe reselection. The existing fee sources disagree about the case where no signed tier qualifies: the payload description and two ticket resolutions select the largest tier with an alert, while the action contract says fail closed, as an earlier explanation says a former

decision did; the records could not locate that decision. The records preserve both statements and identify the conflict; neither interpretation is authorized until it is answered. The fee estimate and signed tier selection contract holds the exact source text.

A pay-to-anchor (P2A) output permits a separately funded child-pays-for-parent (CPFP) child. Product export, deterministic unsigned construction, Core's manually unlocked signing/finalization/broadcast, and independent observation stay separate so a fee wallet gains no participant authority. The CPFP request, local helper, and Bitcoin Core contract owns the arithmetic and operator process. A relay timeout is ambiguous, not permission to send a different transaction; durable exact-attempt reconciliation and independent source agreement govern retry. Wrong binding, stale bytes, contested facts, missing primary archive evidence, or invalid route blocks the affected action.

The recorded rationale for exact scripts and witnesses is that a mismatch can strand collateral or authorize the wrong party. Fixed envelope and key derivation function (KDF) interoperability makes encrypted backups portable; hardened per-loan derivation avoids index reuse, cross-loan correlation, and expanded compromise. Those are reasons for the existing contracts, not permission to add recovery authority. Automatic organization default broadcast, institutional collateral recovery, nonresponse activation, stronger forgotten-password recovery, and complete cooperative operations remain excluded under the automatic lender-default broadcast, institutional last-resort collateral recovery, borrower-nonresponse activation recovery, stronger participant-Vault recovery, and cooperative settlement operations exclusions, each with its own revisit condition.

The architecture guide shows where this domain fits in the system, and the domain record lists exactly what the domain owns and excludes.

DOMAIN GUIDE

# Base USDC Settlement And Funding

This domain owns what happens to USDC on Base. It gives one loan an authoritative contract record and fixed rules for funding, borrower acceptance, repayment, lender claim, and refund. Bitcoin collateral lives under a separate chain authority. A Base payout can expose a secret that makes a previously authorized Bitcoin transaction usable, but it cannot sign, broadcast, or confirm that transaction.

A **funding vault** is a deployed immutable per-loan address for incoming principal; a **repayment vault** is the corresponding address for debt repayment. Transfers into either are provisional until the named main-contract action finalizes the required exact amount. **Payer-neutral** means that the sender can be any eligible address: sending tokens does not acquire the borrower's or lender's role, change a recipient, or gain a refund right. Fixed recipients come from the approved loan and contract state, not the identity of the payer.

## Settlement And Lender Interaction

**Base loan state and USDC settlement** owns the contract's states, deadlines, calls, named events/errors, exact amounts, and fixed recipients. It registers the funding vault; successful borrower acceptance (the `acceptX` call) atomically pays principal to the fixed borrower recipient and deploys/registers the repayment vault. Failed or conflicting calls revert without a success event or unintended value movement. The contract also provides the exact permissionless fixed-effect finalization, sweep, expiry, and refund calls. Permissionless execution lets another caller pay gas; it does not give that caller control over amount, recipient, or meaning. The Base transition interface, events, and errors, the funding vault contract, and the repayment vault contract retain the full public contracts.

**Lender-controlled funding-vault interaction** supplies the human and application boundary for the selected lender's explicit capital action. It presents the finalized deployed vault, chain, asset, loan, remaining principal, deadline, and fixed lender settlement recipient. The lender may use one or more ordinary transfers from a controlled wallet or treasury, including a recoverability-qualified Vault payment account. Partial deposits accumulate. This responsibility coordinates provisional balance display and finalization while leaving signing to the participant and authoritative transitions to the contract and observer. It never pools capital, substitutes a lender, or automatically draws funds.

## From Funding To Repayment

After qualified origination, the main contract opens the funding window. Exact-principal finalization produces a funded-but-unaccepted state with a bounded acceptance lifetime. The borrower must intentionally disclose `X` through its Vault-controlled call; acceptance pays principal and leaves Bitcoin activation pending. Only separate confirmed Bitcoin evidence can

make the product's active-collateral projection eligible. If the borrower never accepts, fixed-recipient expiry/refund remains possible and prefunded Bitcoin has its own recovery path. The origination flow, the activation flow, the preactivation unwind flow, and the main Base loan states preserve these distinctions.

For repayment, sufficient vault balance is necessary but is not itself valid repayment. Exact debt must be finalized no later than `D`. A bound lender can then claim by revealing `Y` strictly before `C = D + 172,800 seconds`. The claim pays the fixed lender recipient once and exposes `Y`; the observation domain qualifies its finality, and Bitcoin operations can complete the fixed borrower recovery transaction. The borrower may make the one permitted refund-recipient switch before `C`; at `C` the current recipient freezes. The repayment vault states, the refund recipient and claim cutoff states, and the normal repayment flow own the exact conditions.

If claim does not happen before `C`, the lender permanently loses that Base claim and the exact unclaimed repayment becomes permissionlessly refundable once to the frozen borrower recipient. The finalized `UnclaimedRepaymentRefunded` event is the only positive Base fallback trigger. Lender silence, a missing secret, elapsed time, or raw token activity is not that event. Base refund does not imply Bitcoin recovery, and Bitcoin lender-default maturity after `T = D + 259,200 seconds` remains independent. The valid repayment, claim and refund exclusivity, and default timing and Base separation invariants keep these consequences separate.

## Failure, Authority, And Scope

Wrong loan, hash, role, recipient, amount, deadline, state, or deployment blocks the action. Excess and late configured USDC follow the specified fixed recovery paths; there is no generic asset rescue or arbitrary destination. Application retries must reload authoritative facts and preserve the same fixed effect. A workforce operator can request an already authorized operation but cannot edit contract meaning or claim success without evidence.

The recorded construction choices use immutable ownerless vaults, while the main contract retains its explicit Transparent Proxy administration, compatibility checks, and opener-replacement boundary. Who holds that administration is an unresolved source conflict: the Base upgrade boundary states that V1 retains it under a 2-of-3 Safe through a seven-day timelock, while the upgrade governance selection states a dedicated single upgrade key for local/test use and the intentionally short public mainnet beta, followed by transfer of that sole ownership to a 2-of-3 multisig-controlled timelock. Neither is chosen until the conflict is answered. Whether V1 has an on-chain pause is also open: this domain's settlement responsibility and the upgrade governance selection say V1 has no on-chain pause, while the deployment, upgrade, and opener states include pause roles and a narrow operational pause; whether that pause is the frontend/API operational stop or an on-chain role is undecided. Direct valid exits remain available during application or opener shutdown. These choices remain governed by the deployment, upgrade, and opener states, the funding opener and pause boundaries, and the Base upgrade boundary. A Vault payment account and the separately scoped participant-transfer capability do not change payer-neutral contracts or historical recipients. Cooperative Bitcoin

settlement does not yet provide a complete Base accounting capability (cooperative settlement operations and Base accounting); liquidation, organization custody, and automatic lender-capital allocation remain outside this domain.

The architecture guide shows where this domain fits in the system, and the domain record lists exactly what the domain owns and excludes.

DOMAIN GUIDE

# Base Observation And Fallback Review

This domain makes evidence from one chain safe to use elsewhere in a Bitcoin-backed USDC loan. The Base contracts settle the USDC; Bitcoin independently controls the collateral. A transaction appearing in a wallet, an updated application screen, or a missing lender response cannot bridge those authorities. This domain supplies trustworthy observations of what happened on Base and defines a narrowly limited fallback that may help return Bitcoin after an unclaimed repayment has been refunded.

It contains two distinct responsibilities. Base observation, finality, and reconciliation follows the approved deployment, records its events and their finality, and repairs the application's view when evidence changes. The evidence-conditioned validation-service fallback can publish a loan's precommitted secret after independently verifying one exact refund event. Observation reports facts; the fallback controls that secret's disclosure. Neither responsibility decides who deserves the collateral or signs a Bitcoin transaction.

## Establishing a fact the rest of the system can use

The observer reads named contract events for funding, acceptance, repayment, claim and refund. It checks which chain, contract deployment, loan, transaction and block produced each event. An event first seen in a Base block is provisional. Even the intermediate “safe” status is only progress information: application-side lifecycle advances and Bitcoin handoffs wait until the block reaches **Base L1 (layer 1) finality**, the stronger settlement condition defined in the finality decision. That decision accepts additional waiting in return for one consistent rule at consequential cross-chain boundaries; it is not a promise of an exact waiting time.

The observer sends those facts to the application through a versioned event-ingestion handoff. The application records the source evidence and derives the status a participant sees. It acknowledges storage before the observer advances its durable cursor—the saved position from which it can safely resume. If an acknowledgement is lost, the observer sends the same event again; the stable identity prevents a duplicate. This event history was selected so that a restart or provider change can reconstruct what happened, instead of trusting a mutable loan snapshot or an independently edited watcher status.

Alchemy is the primary Base read source and QuickNode is the fallback. One healthy configured provider is sufficient for a finalized Base fact; the second provider supplies failover and disagreement detection. If both are unavailable or disagree, existing finalized projections remain visible while new consequential actions that depend on Base are blocked. When a chain reorganization displaces an observed event, the observer retains it as noncanonical evidence, appends a correction, and replays from the last common finalized block. An operator may request a recheck through the application, but cannot change the fact, override the result or advance the observer's cursor.

The responsibility also reconciles independently sourced Bitcoin evidence. Bitcoin lifecycle gates require fresh agreement between the configured observers, unlike the one-healthy-source Base policy. Conflicts and outages block dependent automation; repeated reports update the same incident, recovery replays from the last agreed state, and a failure after borrower acceptance receives immediate critical treatment. Vault payment transfers have another limited meaning: exact transaction and nonce evidence can update a payment's own record, but only the named finalized contract event can establish loan funding or repayment. Even a confirmed standalone participant transfer does not advance the loan. The observer's precise rules retain the provider, retry, escalation and payment-profile distinctions.

## When the borrower repays but the lender does not claim

A concrete case explains why the fallback is separate. Assume exact repayment has finalized by the repayment deadline, **D**. Normally the lender claims the USDC by revealing a secret called **Y** before the claim cutoff. Public Y then lets the Bitcoin finalizer complete a recovery transaction that the borrower already signed for a fixed destination. If the lender cannot or does not reveal Y, valid repayment alone does not release the Bitcoin.

At **C = D + 172,800 seconds**, two days after the repayment deadline, Base permanently disables the lender claim and freezes the current borrower refund recipient. Anyone may call the fixed-effect refund; the caller cannot choose where the money goes. A borrower concerned about access to the original recipient may have switched it to the committed borrower protocol account before C; the choice is frozen at C. Once the refund's exact **UnclaimedRepaymentRefunded** event reaches Base L1 finality, the observer can report that the exact USDC repayment was returned once. The unclaimed-repayment flow makes claim and refund mutually exclusive on Base. The refund does not require the fallback service, and the Bitcoin output remains separately unspent.

The selected fallback is an **evidence-conditioned validation service**, or **ECVS**: one contracted independent service that holds a unique random secret **S** for each eligible loan. Before that loan is constructed and signed, it returns only **H(S)**, a hash commitment used later to check the disclosed secret. The Bitcoin recovery script must already contain this commitment. The company, borrower and lender do not receive unrevealed S and cannot command an early release.

The ECVS does not accept the application observer's conclusion as authority. It independently checks the configured Base deployment, loan, exact refund event, amount, frozen recipient, provenance and L1 finality. Only then may it publish the matching S durably and publicly. Bitcoin finalization separately checks the publication, commitment, existing signatures and still-unspent escrow, then uses S in the **same borrower-signed recovery transaction** that could have used Y. No fresh borrower signature, second fallback transaction family, service signing key or new destination is introduced. Publication, broadcast and confirmed Bitcoin recovery are separate steps; the conditional recovery flow and fixed Bitcoin handoff preserve those boundaries.

## Limits and remaining recovery paths

This model is selected, but production use remains gated by counsel approval and implementation evidence. Until that gate is satisfied, a new loan must not commit H(S) or advertise this fallback. An existing Y-only escrow cannot acquire the capability later. The selected design avoids giving the service a Bitcoin key or discretionary evidence-arbitration role; its secret can still be disclosed too early, withheld, lost or compromised. Those risks are why independent controls and the activation gate remain material.

Missing Y, elapsed time, an application assertion, a raw token transfer or the earlier **Repaid** event cannot trigger S publication. If the service is unavailable, evidence is wrong, or S cannot be published, the Base refund remains complete and Bitcoin recovery through S remains unavailable. The lender's pre-signed Bitcoin route can still mature at D plus three days under Bitcoin's consensus time rules. A separate borrower long-stop can mature at D plus ten days without Y or S, but does not cancel the already-mature lender route; one confirmed spend wins. Whether a loan without a production-activated S commitment has that long-stop is an open question: a build constraint requires every newly constructed escrow to include it and the long-stop recovery flow defines it, while the missing-Y security boundary says such a loan has no borrower fallback Bitcoin route and the unclaimed-repayment flow lists the remaining routes without it; the records do not settle whether these describe different loans or contradict each other. Mutual cooperation is another separately authorized route. These are residual Bitcoin risks, not evidence of borrower default after valid repayment or a guarantee that collateral will return.

Automatic organization broadcast of lender-default packages and institutional collateral recovery remain outside the selected operating boundary. Monitoring, package storage and the ECVS give the company no collateral key, sale authority or discretionary payout power. Exact contracts and the two responsibility records linked above remain the owners of every rule summarized here.

The architecture guide shows where this domain fits in the system, and the domain record lists exactly what the domain owns and excludes.

DOMAIN GUIDE

# Participant Access And Loan Operations

This domain turns the separate agreement, Vault, Base, and Bitcoin authorities into a usable product workflow. It records who may act, what the application durably accepted, which public operation is pending, and which externally sourced result is known. It also owns a deliberately isolated developer demonstration. Application records coordinate those authorities; they cannot become private keys, participant consent, or chain truth.

A **workspace** is the durable application view of a loan. A **revision** identifies the exact state a command was based on. An **outbox** records required work atomically with the state change so a lost immediate wake cannot lose the operation. A **handoff** grants a particular participant access to one bounded Vault action; it is not a general signing token. A **payment intent** fixes one exact public request before the participant's Vault signs. These concepts distinguish an accepted request from a submission, observation, finalized contract event, or confirmed Bitcoin spend.

## Access, Coordination, And Local Composition

**Participant identity and role access** owns accounts, role membership, sessions, recovery, reauthentication and multi-factor authentication (MFA), and permission to retrieve bound opaque artifacts. Product authentication uses the selected provider adapter while stable accounts and loan authorization remain application-owned. Workforce operator access is separate. Authentication can authorize ciphertext retrieval; it cannot decrypt a Vault, recover a seed or secret, or make a chain action valid. The identity, session, and fresh access states and the access contracts retain exact denial and recovery behavior.

**Loan workspace and application coordination** owns revisioned commands, idempotency, durable outbox work, source records, projections, notifications, package archives, sponsorship records, and exact participant handoffs. A consumer can acquire and refresh its authorized workspace, distinguish absent, forbidden, stale, and unavailable results, obtain an accepted-command acknowledgement, and recover the same result after refresh or restart. A mock, health route, or producer-local method does not prove that public consumer path. The workspace command, revision, outbox, and result contract and the workspace command, outbox, and projection states preserve the exact routes and failures.

**Local runtime composition and controlled demonstration** assembles the existing independently built units through their public seams. Its one developer entrypoint preflights, starts or resumes, health-gates, and reports the system; paired status, shutdown, and explicit reset/reseed actions make state ownership clear. The local profile uses loopback Base Anvil, fake USDC, two connected persistent Bitcoin regtest nodes, two isolated participant sessions/Vaults, the real Product/Vault/API/observer/operations paths, and identity-bound fixtures. A coordination descriptor references producer-owned state instead of owning or recursively

deleting sibling repositories. Ordinary shutdown preserves that state; only explicit bounded reset invokes each producer's reset contract. The local demo runtime control contract and the local demo runtime states own the lifecycle.

## Cooperation Through A Loan

Core first binds authenticated participants to a reserved loan and current terms candidate. The dedicated Vault returns acknowledged public identity and preparation material; under the terms domain's detailed rules both roles approve the same final candidate, which is one side of an unresolved source conflict over whether the lender approves again after reservation, explained in the loan terms guide. Bitcoin operations produce complete unsigned signing context, Core stores an immutable batch, Product launches the bounded Vault handoff, and the Vault signs locally. Core revalidates the result, writes the full package archive, and only then issues its receipt. After qualified funding and deliberate borrower acceptance, observed Base `X` and independent Bitcoin evidence drive separate projection and operation steps. Normal repayment similarly waits for finalized contract outcomes and confirmed Bitcoin recovery. The origination flow, the activation flow, and the normal repayment flow provide the authoritative sequence.

A required state change and its job commit together in the D1 workspace database. Immediate private wake is only an optimization; a one-minute scan recovers missed or crashed work. Exact request, route, participant, amount, nonce, fees, bytes, provider, and transaction identities survive ambiguous outcomes. Consumers do not turn a lost response into a new payment or changed action. Known inclusion delay remains distinguishable from unresolved reconciliation; the bounded policy probes at 0, 15, and 60 seconds and escalates at 180 seconds to the declared visible block/recovery route. The Vault-owned payment account and exact transfer contract and the payment account and intent states retain the exact phase, fencing, and evidence contract.

## Payment Authority And Availability

A Vault payment account is independent of its protocol externally owned account (EOA) and per-loan Bitcoin keys. It becomes public only after snapshot and backup recoverability establish same-address restore. Core owns the public intent, lease revision, fence, atomic phase transitions, and provider gate; the Vault owns encrypted pending bytes and signing. Core enters durable Signing before invoking the signer. Public hash and broadcast-phase persistence must occur in their specified order; repeat broadcast uses identical bytes, and changes require fresh review. A timeout after Signing is not proof of non-submission. A new device must restore or verifiably reproduce the exact pending bytes, otherwise the account remains explicitly blocked. None of this changes historical terms, payer-neutral settlement, or observation authority.

The selected narrow participant-transfer purpose permits configured USDC to a validated user-selected recipient with exact approval and balance/gas checks. It has no invented loan/workspace binding and cannot advance loan state. Arbitrary calls, approvals, swaps, other assets, native-value sends, and generic signing remain excluded. Local setup may provision disposable funds to public payment addresses, but that is neither participant-payment evidence nor production gas sponsorship. The latter remains an excluded ticketed capability.

All selected V1 application artifacts share Cloudflare's provider failure domain, despite separate origins, projects, credentials, and releases. A provider-wide outage can suspend the Product, Vault, API/D1, observers, and automation together. The chains continue; new consequential application commands stop. Return requires canonical Base/Bitcoin reconciliation before idempotent jobs resume. Off-provider archive backups protect data recovery, not a running-service fallback or promised recovery time or recovery point objective (RTO/RPO).

## Recorded Choices And Scope

The public-consumer continuity work was needed because a producer-local command and passing harness alone do not ensure a browser can acquire state or recover an accepted operation. The local composition was added because independently implemented units did not by themselves supply a persistent runnable demonstration or safe profile separation. Vault-owned payment intent prevents unrecoverable exposed addresses, duplicate payment after lost responses, and accidental mixing of protocol keys with payer identities. The exact mechanics stay with their contracts; prior implementation repair history stays with tickets and handoffs.

Verification, runnable local use, testnet/mainnet deployment, and activated production operation remain distinct claims under the runtime profile claims and activation gates. The records disagree about the current payment capability's status: the payment account selection says its implementation and execution evidence remain gated, and the browser acceptance verification calls it planned and unverified, while the core coordination handoff records historical payment implementation and the handoff sequence records a correction that now passes focused checks without finalized completion receipts. Which status holds is undecided; the most limited reading is that the capability is selected but planned and unverified. Forgotten-password recovery and complete cooperative-settlement operation remain deferred under the stronger participant-Vault recovery and cooperative settlement operations exclusions. A visual, fixture, healthy process, or local design check cannot satisfy those exclusions or activation gates.

The existing future provider-replacement question remains deferred: WorkOS is the selected V1 identity provider, and a replacement must preserve internal account and immutable loan/Vault identities while proving authenticated linking, session/MFA/recovery parity, cutover, rollback and adapter conformance (application-owned authentication replacement). Optional platform-paid reveal relaying/paymaster execution is also deferred; current support is bounded protocol-EOA prefunding plus participant signing and direct submission (platform-paid Base reveal execution). These are existing exclusions, not newly selected capabilities.

The architecture guide shows where this domain fits in the system, and the domain record lists exactly what the domain owns and excludes.

## Technical records

The exact rules, contracts and conditions live in these records of the design package:

- **Architecture record:** Bitcoin-Backed USDC One-Loan System Architecture.
- **Domain records:** Base Observation And Fallback Review; Base USDC Settlement And Funding; Bitcoin Authorization And Recovery; Loan Terms And Commitments; Participant Access And Loan Operations.
- **Responsibility records:** Base Loan State And USDC Settlement; Base Observation, Finality, And Reconciliation; Bitcoin Prefund, Escrow, And Package Construction; Borrower Bitcoin Authorization And Recovery; Evidence-Conditioned Validation-Service Fallback; Lender Bitcoin Authorization And Recovery; Lender-Controlled Funding-Vault Interaction; Loan Terms, One-Loan Offer, Reservation, And Parameter Binding; Loan Workspace And Application Coordination; Local Runtime Composition And Controlled Demonstration; Participant Identity And Role Access.
- **Flow records:** Activation Completion And Active Entry; Borrower Long-Stop Recovery; Conditional ECVS Borrower Recovery; Normal Repayment Close; Origination Preparation And Qualification; Preactivation Unwind; Valid Repayment Without Lender Claim.
- **Interface contracts:** Base Transition API, Events, And Errors; BaseEventObservationV1 And Correction; Bitcoin Fee Estimate And Signed Tier Selection; Cooperative Settlement Package; Finalized X, Y, Or S To Fixed Bitcoin Action; LoanTermsV1 And Canonical Binding; LocalDemoRuntimeControlV1; One-Loan Offer, Reservation, And Loan Binding; OriginationBindingV1, TerminalBindingV1, And Package Manifests; P2A CPFP Request, Local Helper, And Bitcoin Core; Participant-Local Base Calls; Per-Loan Base Funding Vault; Per-Loan Base Repayment Vault; ProtocolDeploymentV1; Vault Envelope And Local Handoff; Vault-Owned Base Payment Account And Exact Transfer; WorkspaceCommandV1, Revision, Outbox, And Result.
- **State contracts:** Base Deployment, Upgrade, And Opener; Base Repayment Vault; Bitcoin Escrow And Terminal Outcome; Bitcoin Package Graph; Bitcoin Prefund And Origination Attempt; Canonical Loan Terms; Local Demo Runtime; Main Base Loan; One-Loan Offer And Reservation; Product Identity, Session, And Fresh Access; Repayment Refund Recipient And Claim Cutoff; Vault-Owned Base Payment Account And Intent; Workspace Command, Outbox, And Projection.
- **Invariants:** Authority Separation; Base Upgrade Boundary; Canonical Loan Binding; Claim, Refund Exclusivity, And Fallback Trigger; Default Timing And Base Separation; Funding Opener And Pause Boundaries; Key, Destination, And Refund Separation; Missing Y Residual Lender Risk; One Terminal Escrow Outcome; Selected Lender Capital Control And No Liquidation; Valid Repayment; Zero-Confirmation Boundary.
- **Security boundary:** Valid repayment plus missing `Y` leaves Bitcoin unsettled.
- **Scope boundaries:** Application-Owned Authentication Replacement; Automatic Lender-Default Broadcast; Borrower-Nonresponse Activation Recovery; BTC Liquidation, Automated Surplus Return, And Shortfall Enforcement; Cooperative Settlement Operations And Base Accounting; Institutional Last-Resort Collateral Recovery; Platform-Paid Base Reveal Execution; Public Or Multi-Lender Matching, Pooling, Custody, And Automatic Capital Deployment; Runtime Profile Claims And Activation Gates; Stronger Participant-Vault Recovery.
- **Constraint record:** Durable Signed Exit Package Archive.
- **Selection records:** Base contract upgrade governance; Base L1 finality and reorg reconciliation; Vault-Owned Participant Payment Account And Local-Demo Realization.
- **Verification record:** Runnable Local-Demo Browser Acceptance.